Handwritten Digit Recognition on MNIST Using Raw Pixel and HOG Features with SVM Classification

Mingwei Zhang
Department of Computer Science
Queens College, City University of New York
Flushing, NY 11367
mingwei.zhang67@qmail.cuny.edu

Abstract

Handwritten digit recognition is a classical image recognition problem that requires converting image data into useful features for classification. This project compares two non-CNN approaches for recognizing handwritten digits from the MNIST dataset using MATLAB. The baseline method flattens each 28×28 grayscale digit image into a 784-dimensional raw pixel feature vector and trains a multiclass Support Vector Machine (SVM). The main method replaces raw pixel features with Histogram of Oriented Gradients (HOG) features, which describe local edge and stroke-direction patterns, while keeping the same SVM classifier. Experiments are conducted using 60,000 training images and the full 10,000-image MNIST test set. Raw Pixel + SVM achieves 94.38% test accuracy, while HOG + SVM improves accuracy to 98.83%. The results show that HOG features perform better than raw pixel intensities for this digit recognition task.

1 Introduction

Handwritten digit recognition is a fundamental problem in image processing and pattern recognition. The goal is to classify an input image of a handwritten digit into one of ten possible classes, from 0 to 9. The MNIST dataset provides a standardized collection of labeled grayscale handwritten digit images, making it suitable for evaluating how different image representations affect classification performance [1].

A key challenge is that the same digit can appear in many writing styles. Digits may vary in stroke thickness, slant, curvature, and position within the image. A classifier using only raw pixel intensities may be sensitive to these variations because it relies directly on the original image layout.

This project compares a raw-pixel baseline with a Histogram of Oriented Gradients (HOG) feature representation. Both methods use the same multiclass SVM classifier, so the comparison focuses on how the feature representation affects handwritten digit recognition.

2 Literature Review

2.1 MNIST Digit Recognition

MNIST is a widely used benchmark dataset for handwritten digit recognition. It contains grayscale digit images with labels from 0 to 9 and has been used to evaluate recognition methods for handwritten characters [1]. In this project, MNIST is loaded in MATLAB using the Lucidar `mnist.mat` format, which provides separate training and test image arrays with corresponding labels [2].

2.2 Histogram of Oriented Gradients

Histogram of Oriented Gradients (HOG) is a handcrafted image descriptor based on local gradient orientation patterns [4]. The main idea is that object shape can be represented by the distribution of local edge directions. This makes HOG useful for digit recognition because handwritten digits are mainly defined by strokes, curves, and edges. MATLAB also provides HOG feature extraction through `extractHOGFeatures`, and its digit-classification example shows HOG features being used with a multiclass classifier [5, 6].

2.3 Support Vector Machine Classification

Support Vector Machines (SVMs) are supervised classifiers that learn decision boundaries with large margins [7]. Since MNIST contains ten digit classes, the classifier must handle multiclass prediction rather than only binary classification. MATLAB's `fitcecoc` supports multiclass classification by using an ECOC model, which reduces a multiclass problem into multiple binary classification problems [3, 8, 9].

3 Methodology

3.1 Overview of the Recognition Pipeline

The goal of the recognition pipeline is to convert each handwritten digit image into a numerical feature vector and then classify that vector into one of ten digit classes. Each MNIST image is a grayscale matrix

$$I \in \mathbb{R}^{28\times 28}, \tag{1}$$

with label

$$y \in \{0, 1, 2, \ldots, 9\}. \tag{2}$$

The general pipeline is:

$$I \to \mathbf{x} \to \hat{y}, \tag{3}$$

where I is the input image, $\mathbf{x}$ is the extracted feature vector, and $\hat{y}$ is the predicted digit label. Two different feature representations are tested. The baseline uses raw pixel intensity values directly, while the main method uses Histogram of Oriented Gradients (HOG) features. Both representations are passed to the same multiclass SVM classifier. This design keeps the classifier fixed and makes the feature representation the main variable being tested.

3.2 Raw Pixel Feature Baseline

The baseline method uses the original pixel values of the image without additional feature extraction. Each 28×28 image is flattened into a one-dimensional vector:

$$\mathbf{x}_{\text{raw}} = \text{vec}(I), \tag{4}$$

where

$$\mathbf{x}_{\text{raw}} \in \mathbb{R}^{784}. \tag{5}$$

The vectorization operation stacks all image pixels into a single feature vector. Each feature corresponds to the grayscale intensity of one pixel. This representation directly preserves the image intensity values.

However, raw pixel features do not explicitly describe edges, curves, or stroke direction. Two images of the same digit may have very different raw pixel vectors if the digit is slightly shifted, slanted, or written with a different stroke thickness. For example, two handwritten 5s may belong to the same class but place their dark pixels in different locations. In the raw pixel baseline, the SVM must learn these shape variations directly from the 784-dimensional intensity space.

This method serves as the baseline comparison point. Since the classifier is kept the same, any performance difference mainly comes from the feature representation rather than the classifier.

3.3 HOG Feature Representation

The main method uses Histogram of Oriented Gradients (HOG) features. HOG is a handcrafted descriptor that represents local shape by measuring the distribution of gradient directions in small image regions [4, 5, 6]. This is suitable for handwritten digit recognition because digits are mainly defined by strokes, edges, and curves.

For an image I, the first step is to estimate the horizontal and vertical image gradients:

$$G_x(x, y) = \frac{\partial I(x, y)}{\partial x}, \qquad G_y(x, y) = \frac{\partial I(x, y)}{\partial y}. \tag{6}$$

These gradients measure how quickly the image intensity changes along the horizontal and vertical directions. Pixels near digit strokes usually produce stronger gradients because the intensity changes sharply between the dark digit and the lighter background.

The gradient magnitude and orientation are then computed as:

$$M(x, y) = \sqrt{G_x(x, y)^2 + G_y(x, y)^2}, \tag{7}$$

$$\theta(x, y) = \operatorname{atan2}(G_y(x, y), G_x(x, y)). \tag{8}$$

The magnitude $M(x, y)$ describes the strength of the edge at pixel (x, y), while the orientation $\theta(x, y)$ describes the direction of that edge. Instead of storing every pixel intensity directly, HOG summarizes how edge directions are distributed across local parts of the image.

The image is divided into small cells. In this experiment, the cell size is set to 4×4 pixels. Since each MNIST image is only 28×28, this cell size allows the descriptor to capture fine local stroke patterns while still grouping nearby pixels into meaningful regions. Within each cell, gradient orientations are accumulated into a histogram. Pixels with larger gradient magnitudes contribute more strongly to the histogram because they represent stronger edges.

Following the HOG representation described in [4, 5, 6], the descriptor for an image can be written as:

$$\mathbf{x}_{\mathrm{HOG}} = \phi_{\mathrm{HOG}}(I), \tag{9}$$

where $\phi_{\mathrm{HOG}}(\cdot)$ denotes the HOG feature extraction process.

Compared with raw pixels, HOG uses edge and shape information instead of only pixel intensity values. This makes the classifier less dependent on exact pixel locations and more focused on local stroke structure. For handwritten digits, this is necessary because the same digit may be written with different thickness, curvature, or slant while still preserving similar edge-direction patterns.

3.4 Support Vector Machine Classification

After feature extraction, both methods use a Support Vector Machine classifier. An SVM attempts to find a decision boundary that separates classes with a large margin [7]. For binary classification, the decision function has the form:

$$f(\mathbf{x}) = \mathbf{w}^T\mathbf{x} + b, \tag{10}$$

where $\mathbf{x}$ is the feature vector, $\mathbf{w}$ is the learned weight vector, and b is the bias term.

The predicted class in binary classification depends on the sign of $f(\mathbf{x})$. A positive value places the sample on one side of the hyperplane, while a negative value places it on the other side. The margin is the distance between the separating hyperplane and the closest training samples. A larger margin generally improves generalization because the classifier is less sensitive to small changes in the input features.

Following the soft-margin SVM formulation in [7], the objective is:

$$\min_{\mathbf{w}, b, \boldsymbol{\xi}} \frac{1}{2}\|\mathbf{w}\|^2 + C\sum_{i=1}^{N} \xi_i, \tag{11}$$

subject to

$$y_i(\mathbf{w}^T\mathbf{x}_i + b) \geq 1 - \xi_i, \qquad \xi_i \geq 0. \tag{12}$$

The first term, $\frac{1}{2}\|\mathbf{w}\|^2$, encourages a larger margin. The slack variables ξ_i allow some samples to fall inside the margin or be misclassified. The parameter C controls the tradeoff between margin size and classification error. A larger C penalizes training errors more strongly, while a smaller C allows more margin violations.

MNIST is a ten-class problem, so a binary SVM must be extended to multiclass classification. MATLAB's `fitcecoc` handles this using error-correcting output codes (ECOC), where multiple binary classifiers are trained and combined to predict one of the ten digit labels [3, 8, 9]. The final prediction is:

$$\hat{y} \in \{0, 1, 2, \ldots, 9\}. \tag{13}$$

Using the same SVM framework for both Raw Pixel + SVM and HOG + SVM ensures that the comparison is controlled. The experiment does not test different classifiers; it tests whether changing the image representation improves recognition.

3.5 Algorithm Summary

The complete procedure is summarized below:

1. Load each MNIST image I and its label y.
2. For the baseline method, convert I into $\mathbf{x}_{\text{raw}}$ by flattening the 28×28 image into a 784-dimensional vector.
3. For the main method, compute image gradients, form local orientation histograms, and convert I into $\mathbf{x}_{\text{HOG}}$.
4. Train a multiclass SVM using the selected feature representation.
5. Predict labels for the test images using the trained classifier.
6. Compare predicted labels with ground-truth labels using accuracy, per-class accuracy, and confusion matrices.

This methodology compares raw intensity information with handcrafted gradient information. The raw pixel baseline tests whether the SVM can classify digits from the original image layout alone, while the HOG method tests whether edge and stroke-direction features improve handwritten digit recognition.

4 Experimentation

4.1 Dataset and Setup

The experiments use the MNIST dataset loaded from the Lucidar MATLAB `mnist.mat` format [2]. The dataset contains 60,000 training images and 10,000 test images. Each image is a 28×28 grayscale handwritten digit, and each label belongs to one of ten classes from 0 to 9. In MATLAB, the image data is stored as three-dimensional arrays, where the first two dimensions represent image height and width and the third dimension indexes the image sample:

$$\texttt{training.images} \in \mathbb{R}^{28\times28\times60000}, \qquad \texttt{test.images} \in \mathbb{R}^{28\times28\times10000}. \tag{14}$$

Before training, the dataset structure is verified by checking the image and label dimensions. Random training samples are also displayed to confirm that the loaded images visually match their corresponding labels. This step is important because the project depends on correct alignment between image arrays and label vectors. If image-label alignment is incorrect, the classifier may train successfully but learn meaningless mappings.

The final experiment uses 60,000 training images and all 10,000 test images:

$$N_{\text{train}} = 60000, \qquad N_{\text{test}} = 10000. \tag{15}$$

Both the raw pixel baseline and the HOG-based method use the same full training set and the same full test set. This keeps the comparison fair because the only major difference between the two pipelines is the feature representation.

4.2 Feature Extraction Procedure

Two feature extraction procedures are evaluated. For the raw pixel baseline, each 28×28 image is reshaped into a 784-dimensional row vector. The resulting training feature matrix has the form:

$$X_{\text{raw}} \in \mathbb{R}^{60000 \times 784}. \tag{16}$$

The same reshaping procedure is applied to the test images before prediction.

For the HOG-based method, HOG features are extracted from each image using a cell size of 4×4 pixels. A single sample image is first used to determine the HOG feature length. After that, feature matrices are allocated for the full training and test sets. Each image is then converted from its original pixel representation into a HOG descriptor. This produces:

$$X_{\text{HOG}} \in \mathbb{R}^{60000 \times d}, \tag{17}$$

where d is the length of the HOG feature vector produced by the selected HOG settings.

A 4×4 cell size is used because MNIST images have a limited spatial resolution of 28×28 pixels. Larger cells may lose fine stroke information, while smaller cells preserve more local edge structure. Since handwritten digits are made of short strokes, curves, and corners, the selected cell size allows the HOG descriptor to capture detailed local shape patterns.

4.3 Training and Testing Procedure

Both pipelines follow the same training and testing process. First, features are extracted from the selected 60,000 training images. These feature vectors and their corresponding labels are used to train a multiclass SVM classifier. Then, the same feature extraction method is applied to the 10,000 test images. The trained SVM predicts one label for each test sample.

The testing process can be written as:

$$\hat{y}_i = f(\mathbf{x}_i), \tag{18}$$

where $\mathbf{x}_i$ is the feature vector for the i-th test image, $f(\cdot)$ is the trained multiclass SVM model, and $\hat{y}_i$ is the predicted digit label.

The same classifier type is used for both methods. This means the experiment is not comparing two different learning algorithms. Instead, it compares how well the same classifier performs when given two different image representations. This is important because it allows the result to be interpreted as evidence about the feature representation itself.

No separate hyperparameter tuning was performed. Both methods use MATLAB's `fitcecoc` multiclass classification function with the same settings. The HOG cell size was fixed at 4×4, and the 10,000-image MNIST test set was used only for final evaluation.

4.4 Validation / Testing Process and Performance Metrics

The final evaluation is performed on the 10,000-image MNIST test set. Since the dataset already provides a separate test split, performance is measured by comparing the predicted test labels against the ground-truth test labels. The main metric is overall classification accuracy:

$$\text{Accuracy} = \frac{1}{N_{\text{test}}} \sum_{i=1}^{N_{\text{test}}} \mathbb{1}(\hat{y}_i = y_i) \times 100\%. \tag{19}$$

Here, $\hat{y}_i$ is the predicted label, y_i is the true label, and $\mathbb{1}(\cdot)$ equals 1 when the prediction is correct and 0 otherwise.

A confusion matrix is also computed for each method to summarize true and predicted digit labels [10]. Let $C_{i,j}$ denote the number of test samples whose true class is i and predicted class is j. Correct predictions appear on the diagonal entries $C_{i,i}$, while off-diagonal entries show misclassifications. The confusion matrix is useful because it shows not only how many mistakes are made, but also which digit classes are confused with each other.

Per-class accuracy is computed from the confusion matrix:

$$\text{Accuracy}_c = \frac{C_{c,c}}{\sum_{j=0}^{9} C_{c,j}} \times 100\%. \tag{20}$$

This metric is important because overall accuracy alone may hide class-specific weaknesses. For example, a classifier may perform well on easier digits such as 0 and 1 but still struggle with digits that have more variable shapes, such as 5, 8, or 9.

4.5 Results

Figure 1 shows random MNIST training samples. The samples confirm that the dataset contains centered grayscale handwritten digits with visible variation in writing style.

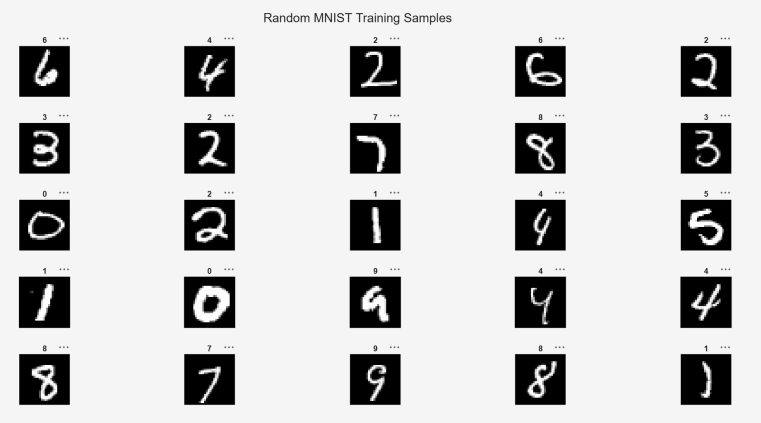


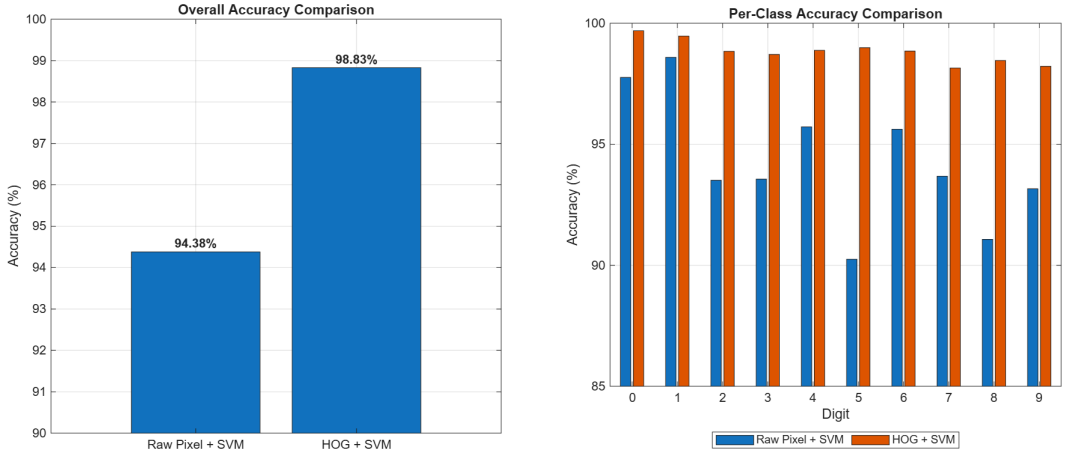
Figure 1: Random MNIST training samples with corresponding labels.

Raw Pixel + SVM gets 94.38% test accuracy, while HOG + SVM gets 98.83%. The difference is $98.83\% - 94.38\% = 4.45\%$.

Table 1: Overall Accuracy Comparison

Method	Test Accuracy
Raw Pixel + SVM	94.38%
HOG + SVM	98.83%

Since both pipelines use the same SVM classifier and the same train-test split, the improvement is mainly due to the change from raw intensity features to gradient-based HOG features. Figure 2 displays both the overall accuracy and the per-class accuracy comparison.



(a) Overall accuracy

(b) Per-class accuracy

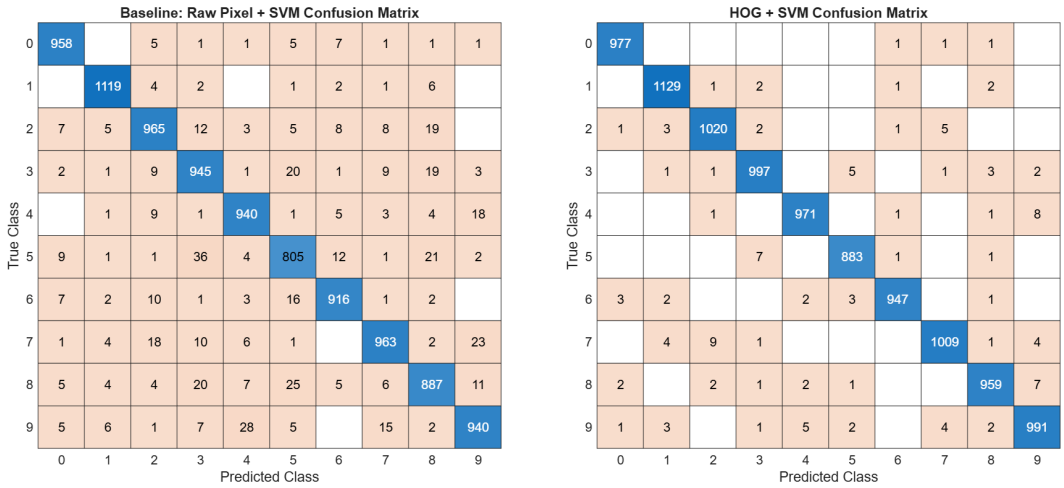
Figure 2: Accuracy comparison between Raw Pixel + SVM and HOG + SVM.

Table 2: Per-Class Accuracy Comparison

Digit	Raw Pixel + SVM	HOG + SVM
0	97.76%	99.69%
1	98.59%	99.47%
2	93.51%	98.84%
3	93.56%	98.71%
4	95.72%	98.88%
5	90.25%	98.99%
6	95.62%	98.85%
7	93.68%	98.15%
8	91.07%	98.46%
9	93.16%	98.22%

The per-class results show that HOG + SVM improves accuracy for every digit class. The largest improvements occur for digits 5, 8, and 9. These digits often contain loops, curves, and variable stroke patterns, making them more difficult to represent using raw pixel intensities alone. HOG reduces this weakness by describing local edge directions instead of relying only on exact pixel locations.

The confusion matrices in Figure 3 provide a more detailed view of the classification errors. The raw pixel baseline has more off-diagonal entries, indicating more confusion between similar digit classes. The HOG + SVM matrix has stronger diagonal values, showing that more test samples are correctly assigned to their true classes.



(a) Raw Pixel + SVM

(b) HOG + SVM

Figure 3: Confusion matrices for the two methods.

Overall, the results show that HOG features perform better than raw pixel features on this MNIST classification task. The improvement is visible in the overall accuracy, the per-class accuracy table, and the confusion matrices.

5 Discussion and Conclusion

5.1 Discussion

The results show that feature representation matters even when the classifier remains unchanged. Raw Pixel + SVM performs well overall, but it is more affected by handwriting variation because it uses intensity values directly. This is most visible for digits such as 5, 8, and 9, where curves and loops vary across writers.

HOG + SVM performs better because it represents local gradient orientation patterns instead of only raw pixel intensities. Since handwritten digits are mainly made of strokes and edges, HOG features match the digit shapes better than raw pixel values. The remaining HOG errors are likely caused by ambiguous handwriting, such as 7s resembling 1s or 9s resembling 4s.

5.2 Limitations and Future Work

This experiment is limited to classical feature extraction and SVM classification. CNNs are not evaluated because the project focuses on image-processing feature extraction rather than automatically learned features. Future work could test preprocessing methods such as centering, deskewing, normalization, or noise reduction. Other classical classifiers, such as k-nearest neighbors, random forests, or logistic regression, could also be compared with SVM using the same feature sets.

5.3 Conclusion

Raw Pixel + SVM and HOG + SVM were compared for MNIST handwritten digit recognition. The raw pixel baseline reaches 94.38% test accuracy, while HOG + SVM reaches 98.83%. HOG improves accuracy for every digit class, showing that edge-based HOG features work better than raw pixel values for this task.

Appendix: Individual Contribution

This project was completed individually by Mingwei Zhang. The work included dataset loading, feature extraction, SVM training, evaluation, result visualization, and report writing.

References

[1] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner. (1998). Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11), 2278–2324.

[2] Lucidar. "Load MNIST database of handwritten digits in Matlab." [Online]. Available: `https://lucidar.me/en/matlab/load-mnist-database-of-handwritten-digits-in-matlab/`

[3] MathWorks. "ClassificationECOC: Multiclass model for support vector machines and other classifiers." [Online]. Available: `https://www.mathworks.com/help/stats/classificationecoc.html`

[4] N. Dalal and B. Triggs. (2005). Histograms of oriented gradients for human detection. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*.

[5] MathWorks. "Digit Classification Using HOG Features." [Online]. Available: `https://www.mathworks.com/help/vision/ug/digit-classification-using-hog-features.html`

[6] MathWorks. "extractHOGFeatures: Extract histogram of oriented gradients features." [Online]. Available: `https://www.mathworks.com/help/vision/ref/extracthogfeatures.html`

[7] C. Cortes and V. Vapnik. (1995). Support-vector networks. *Machine Learning*, 20, 273–297.

[8] MathWorks. "Support Vector Machine Classification." [Online]. Available: `https://www.mathworks.com/help/stats/support-vector-machine-classification.html`

[9] MathWorks. "fitcecoc: Fit multiclass models for support vector machines or other classifiers." [Online]. Available: `https://www.mathworks.com/help/stats/fitcecoc.html`

[10] MathWorks. "ConfusionMatrixChart Properties." [Online]. Available: `https://www.mathworks.com/help/stats/mlearnlib.graphics.chart.confusionmatrixchart-properties.html`